

Chapter 5 Cleaning Code for Cost Data

This chapter walks you through all of the R code used to clean the raw **COST**-related data inputs. The resulting cleaned dataset is then used to calculate the cost impact factors (see **Chapter XXX**).

5.1 Clean and Restructure Intermediary Datasets

This script imports all of raw cost-related data inputs, and then cleans and merges them so that the cost impact factors can later be calculated.

Note that you must first open the 'methods_manual' R project before running this script or else it will not work.

First, let's set up our environment.

```
# check working directory  
getwd()
```

```
## [1] "/Users/bmb73/Library/CloudStorage/Box-Box/lasting_aim_3/model development/methods_n
```



```
# SET UP -----  
  
rm(list = ls())  
  
options(scipen=999)  
  
library(tidyverse)  
library(readxl)  
library(survey)  
  
my_date <- Sys.Date()
```

5.1.1 Import and Merge Data Inputs

NHANES Diet Intake

First, read in the NHANES food-level datasets. The data were split into “day 1” and “day 2” earlier, and here we are going to merge them into “both days”. There also are different datasets containing just sugar sweetened beverage (SSB) data, so we have to import those in separately, and then merge with the rest of the day1/day2 diet datasets.

Import all of the “day 1” datasets first.

```

# import
day1 <- read_rds("data_inputs/DIET/dietary_intake/DATA/clean_data/foods_day1_clear

# only select needed variables
day1_sub <- day1 %>% select(SEQN, DRDINT, DR1DRSTZ, DR1ILINE, DR1IFDCD,
                           DR1IGRMS, DESCRIPTION, foodsource, nhanes_cycle, dayre
  rename(seqn = SEQN,
         line = DR1ILINE,
         foodcode = DR1IFDCD,
         grams = DR1IGRMS,
         description = DESCRIPTION,
         daysintake = DRDINT,
         reliable = DR1DRSTZ)

# ssb
ssb_1 <- read_rds("data_inputs/DIET/dietary_intake/DATA/clean_data/foods_day1_ssb.

# merge food and ssb
day1_sub1 <- left_join(day1_sub, ssb_1, by = c("seqn" = "SEQN",
                                              "line" = "DR1ILINE",
                                              "foodcode" = "DR1IFDCD",
                                              "description" = "DESCRIPTION",
                                              "grams" = "DR1IGRMS"))

```



Then import all of “day 2” datasets.

```

# import
day2 <- read_rds("data_inputs/DIET/dietary_intake/DATA/clean_data/foods_day2_clear

# only select needed variables
day2_sub <- day2 %>% select(SEQN, DRDINT, DR2DRSTZ, DR2ILINE, DR2IFDCD,
                           DR2IGRMS, DESCRIPTION, foodsource, nhanes_cycle, dayre
  rename(seqn = SEQN,
         line = DR2ILINE,
         foodcode = DR2IFDCD,
         grams = DR2IGRMS,
         description = DESCRIPTION,
         daysintake = DRDINT,
         reliable = DR2DRSTZ)

# ssb
ssb_2 <- read_rds("data_inputs/DIET/dietary_intake/DATA/clean_data/foods_day2_ssb.

# merge food and ssb
day2_sub1 <- left_join(day2_sub, ssb_2, by = c("seqn" = "SEQN",
                                              "line" = "DR2ILINE",
                                              "foodcode" = "DR2IFDCD",
                                              "description" = "DESCRIPTION",
                                              "grams" = "DR2IGRMS"))

```

Then, combine the “day 1” and “day 2” datasets to get a “both_days” dataset.

```

both_days <- rbind(day1_sub1, day2_sub1) %>% arrange(seqn, dayrec, line)

# check to make sure the formatting worked
both_days %>% filter(description == "Meat, NFS") %>% head() #looks good!

```

```
## # A tibble: 6 × 12
##   seqn daysintake reliable  line foodcode grams description foodsource nhanes_cycle day
##   <dbl>      <dbl>    <dbl> <dbl>    <dbl> <dbl> <chr>      <chr>      <chr>      <chr>
## 1 84032          1        1     9 20000000 11.0  Meat, NFS  Other      2015-2016
## 2 86178          2        1     5 20000000  4.19 Meat, NFS  Other      2015-2016
## 3 86487          2        1    10 20000000 73.3  Meat, NFS  Grocery    2015-2016
## 4 86801          2        1    13 20000000 11.2  Meat, NFS  Other      2015-2016
## 5 87065          2        1     3 20000000 50.2  Meat, NFS  Other      2015-2016
## 6 88767          2        1    12 20000000  7.33 Meat, NFS  Other      2015-2016
```

Lastly, remove the original diet datasets because they are very large and we don't need them anymore.

```
rm(list=setdiff(ls(), c("both_days", "my_date")))
```

📄 Mappings from Dietary Factor to FNDDS Food Code

There are two mappings from dietary factor to FNDDS Food Code. There is one specifically just for whole and refined grains - see **section XXX** on how the whole vs. refined grains designation was determined. And the second one is for all other non-grain foods.

First, create an empty dataset template that contains all of the foodcodes that exist in the diet dataset (both_days).

```
all_foodcodes <- both_days %>% select(foodcode) %>% distinct()
```

Then, import the non-grain food mapping (labeled as "map_a").

```
map_a <- read_csv("data_inputs/OTHER/dietfactor_to_fndds_mapping/DATA/Food_to_FNDC
```

```
## Rows: 109 Columns: 2
## — Column specification —————
## Delimiter: ","
## chr (1): Foodgroup
## dbl (1): foodcode_prefix
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Then, join the template and the first mapping.

```
my_join <- full_join(mutate(all_foodcodes, i=1),
                     mutate(map_a, i=1)) %>%
  select(-i) %>%
  filter(str_detect(foodcode, paste0("^", foodcode_prefix))) %>%
  select(-foodcode_prefix)
```

```
## Joining with `by = join_by(i)`
```

```
## Warning in full_join(mutate(all_foodcodes, i = 1), mutate(map_a, i = 1)): Detected an un-
## i Row 1 of `x` matches multiple rows in `y`.
## i Row 1 of `y` matches multiple rows in `x`.
## i If a many-to-many relationship is expected, set `relationship = "many-to-many"` to sil
```

Import the grain-only mapping (labeled as “map_b”).

```
map_b <- read_csv("data_inputs/OTHER/dietfactor_to_fndds_mapping/DATA/Food_to_FNDC
```

```
## Rows: 1728 Columns: 2
## — Column specification —————
## Delimiter: ","
## chr (1): Foodgroup
## dbl (1): foodcode
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Merge my_join with the second mapping.

```
my_join1 <- rbind(my_join, map_b) %>% arrange(foodcode)
```

Lastly, merge back with both_days.

```
both_days1 <- left_join(both_days, my_join1, by = "foodcode")
```

Check how many FNDDS codes in the dataset don't have a mapping to a dietary factor (i.e., food group category)

```
both_days1 %>%
  filter(is.na(Foodgroup)) %>%
  select(foodcode, description) %>%
  distinct() %>%
  arrange(foodcode)
```

```
## # A tibble: 0 × 2
## # i 2 variables: foodcode <dbl>, description <chr>
```

Check **SSB** by comparing the “ssb” indicator variable and the data when the foodgroup is set to “ssb”.

```
both_days1 %>% filter(ssb == 1) %>% head()
```

```
## # A tibble: 6 × 13
```

```
##   seqn daysintake reliable line foodcode grams description foodsource nhanes_cycle da
##   <dbl>      <dbl>    <dbl> <dbl>    <dbl> <dbl> <chr>      <chr>      <chr>      <
## 1 83732          2        1   14 92308000 372 Tea, iced, ... Other    2015-2016
## 2 83736          2        1    2 92410510 326. Soft drink,... Grocery  2015-2016
## 3 83736          2        1    7 95320200 356. Sports drin... Grocery  2015-2016
## 4 83736          2        1    5 95320200 372 Sports drin... Grocery  2015-2016
## 5 83736          2        1    7 92410310 248 Soft drink,... Grocery  2015-2016
## 6 83742          2        1   13 92510960 207. Lemonade, f... Other    2015-2016
```

```
both_days1 %>% filter(ssb == 1 & Foodgroup == "ssb") %>% head()
```

```
## # A tibble: 6 × 13
```

```
##   seqn daysintake reliable line foodcode grams description foodsource nhanes_cycle da
##   <dbl>      <dbl>    <dbl> <dbl>    <dbl> <dbl> <chr>      <chr>      <chr>      <
## 1 83732          2        1   14 92308000 372 Tea, iced, ... Other    2015-2016
## 2 83736          2        1    2 92410510 326. Soft drink,... Grocery  2015-2016
## 3 83736          2        1    7 92410310 248 Soft drink,... Grocery  2015-2016
## 4 83742          2        1   13 92510960 207. Lemonade, f... Other    2015-2016
## 5 83742          2        1    9 92410310 150. Soft drink,... Other    2015-2016
## 6 83744          2        1    8 92410510 512 Soft drink,... Other    2015-2016
```

```
both_days1 %>% filter(ssb == 1 & Foodgroup != "ssb") %>% head() # need to change t
```



```
## # A tibble: 6 × 13
```

```
##   seqn daysintake reliable  line foodcode grams description  foodsource nhanes_cycle da
##   <dbl>      <dbl>    <dbl> <dbl>    <dbl> <dbl> <chr>          <chr>      <chr>      <
## 1 83736          2        1     7 95320200  356. Sports drin... Grocery    2015-2016
## 2 83736          2        1     5 95320200  372 Sports drin... Grocery    2015-2016
## 3 83753          2        1     6 95320200  744 Sports drin... Other      2015-2016
## 4 83768          2        1    12 64204010  580. Mango nectar Grocery    2015-2016
## 5 83780          2        1     9 64205010  116. Peach nectar Grocery    2015-2016
## 6 83795          2        1    10 95320200  310 Sports drin... Other      2015-2016
```

```
both_days1 %>% filter(ssb == 0 & Foodgroup == "ssb") %>% head() # need to change t
```

```
## # A tibble: 6 × 13
```

```
##   seqn daysintake reliable  line foodcode grams description  foodsource nhanes_cycle da
##   <dbl>      <dbl>    <dbl> <dbl>    <dbl> <dbl> <chr>          <chr>      <chr>      <
## 1 83732          2        1     1 92101000  720 Coffee, bre... Grocery    2015-2016
## 2 83732          2        1     7 92410320  600 Soft drink,... Grocery    2015-2016
## 3 83732          2        1     1 92101000  255 Coffee, bre... Grocery    2015-2016
## 4 83732          2        1    15 92410520  480 Soft drink,... Other      2015-2016
## 5 83733          2        1     1 92101000  480 Coffee, bre... Grocery    2015-2016
## 6 83733          2        1     1 92101000  450 Coffee, bre... Grocery    2015-2016
```

```
both_days1 %>% filter(ssb == 0 & Foodgroup != "ssb") %>% head()
```

```
## # A tibble: 6 × 13
##   seqn daysintake reliable  line foodcode  grams description foodsource nhanes_cycle dā
##   <dbl>      <dbl>    <dbl> <dbl>   <dbl> <dbl> <chr>      <chr>      <chr>      <
## 1 83732          2        1     2 91200040    4    Sugar subs... Grocery    2015-2016
## 2 83732          2        1     3 12210400    3.92 Coffee cre... Grocery    2015-2016
## 3 83732          2        1     4 94100100 240    Water, bot... Grocery    2015-2016
## 4 83732          2        1     5 51101010   75    Bread, whi... Grocery    2015-2016
## 5 83732          2        1     6 81103080  28.6 Margarine-... Grocery    2015-2016
## 6 83732          2        1     8 27564060 204    Frankfurte... Grocery    2015-2016
```

We see that, for some of the rows, ssb is labeled incorrectly, so we need to fix it.

```
both_days2 <- both_days1 %>%
  mutate(Foodgroup = ifelse(ssb == 1 & Foodgroup != "ssb", "ssb", Foodgroup)) %>%
  mutate(Foodgroup = ifelse(ssb == 0 & Foodgroup == "ssb", "other", Foodgroup))
```

Check again. Looks good.

```
both_days2 %>% filter(ssb == 1 & Foodgroup == "ssb") %>% head()
```

```
## # A tibble: 6 × 13
##   seqn daysintake reliable  line foodcode  grams description foodsource nhanes_cycle dā
##   <dbl>      <dbl>    <dbl> <dbl>   <dbl> <dbl> <chr>      <chr>      <chr>      <
## 1 83732          2        1    14 92308000   372 Tea, iced, ... Other    2015-2016
## 2 83736          2        1     2 92410510   326. Soft drink,... Grocery  2015-2016
## 3 83736          2        1     7 95320200   356. Sports drin... Grocery  2015-2016
## 4 83736          2        1     5 95320200   372 Sports drin... Grocery  2015-2016
## 5 83736          2        1     7 92410310   248 Soft drink,... Grocery  2015-2016
## 6 83742          2        1    13 92510960   207. Lemonade, f... Other    2015-2016
```

```
both_days2 %>% filter(ssb == 1 & Foodgroup != "ssb") %>% head() # none-good
```

```
## # A tibble: 0 × 13
```

```
## # i 13 variables: seqn <dbl>, daysintake <dbl>, reliable <dbl>, line <dbl>, foodcode <db
## #   description <chr>, foodsource <chr>, nhanes_cycle <chr>, dayrec <dbl>, added_sugar <
## #   Foodgroup <chr>
```

```
both_days2 %>% filter(ssb == 0 & Foodgroup == "ssb") %>% head() # none-good
```

```
## # A tibble: 0 × 13
```

```
## # i 13 variables: seqn <dbl>, daysintake <dbl>, reliable <dbl>, line <dbl>, foodcode <db
## #   description <chr>, foodsource <chr>, nhanes_cycle <chr>, dayrec <dbl>, added_sugar <
## #   Foodgroup <chr>
```

```
both_days2 %>% filter(ssb == 0 & Foodgroup != "ssb") %>% head()
```

```
## # A tibble: 6 × 13
```

```
##   seqn daysintake reliable line foodcode grams description foodsource nhanes_cycle da
##   <dbl>      <dbl>    <dbl> <dbl>   <dbl> <dbl> <chr>          <chr>          <chr>          <
## 1 83732         2        1     1 92101000 720   Coffee, br... Grocery      2015-2016
## 2 83732         2        1     2 91200040   4   Sugar subs... Grocery      2015-2016
## 3 83732         2        1     3 12210400  3.92 Coffee cre... Grocery      2015-2016
## 4 83732         2        1     4 94100100 240   Water, bot... Grocery      2015-2016
## 5 83732         2        1     5 51101010  75   Bread, whi... Grocery      2015-2016
## 6 83732         2        1     6 81103080 28.6  Margarine-... Grocery      2015-2016
```

Tidy up the global environment and only keep the datasets we currently need.

```
rm(list=setdiff(ls(), c("both_days2", "my_date")))
```

Food-At-Home (FAH) to Food-Away-Fram-Home (FAFH) Ratios

Import the FAH to FAFH ratios, **whcih** are later used to calculate the price of food purchased at grocery stores (i.e., food at home) vs. food bought **outside of grocery stores** (i.e., food away from home).

```
# import
ratios <- read_csv("data_inputs/ECONOMIC/fah_fafh_ratio/DATA/output_data/fafh_fah_
  select(Diet_var, ratio_FAFH_FAH)
```

```
## Rows: 23 Columns: 5
```

```
## — Column specification —————
```

```
## Delimiter: ","
```

```
## chr (2): Diet_var, Diet_label
```

```
## dbl (3): mean_100g_FAH, mean_100g_FAFH, ratio_FAFH_FAH
```

```
##
```

```
## i Use `spec()` to retrieve the full column specification for this data.
```

```
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# rename "dairy" to "dairy_tot"
```

```
ratios1 <- ratios %>%
```

```
  mutate(Diet_var_new = ifelse(Diet_var == "dairy", "dairy_tot", Diet_var)) %>%
```

```
  select(-Diet_var)
```

Merge with both_days and then check for any missing values.

```

# join with both_days
both_days3 <- left_join(both_days2, ratios1, by = c("Foodgroup" = "Diet_var_new"))

# rename vars
both_days4 <- both_days3 %>%
  rename(Foodgroup_FNDDS = Foodgroup)

# any missing ratios?
both_days4 %>% filter(is.na(ratio_FAFH_FAH)) %>% head()

```

```

## # A tibble: 6 × 14
##   seqn daysintake reliable  line foodcode grams description      foodsource nhane
##   <dbl>      <dbl>    <dbl> <dbl>   <dbl> <dbl> <chr>          <chr>    <chr>
## 1 83732         2        1     4 94100100   240 Water, bottled, unswe... Grocery  2015-
## 2 83732         2        1    11 94100100   240 Water, bottled, unswe... Grocery  2015-
## 3 83732         2        1    14 94100100   360 Water, bottled, unswe... Grocery  2015-
## 4 83732         2        1    16 94000100   120 Water, tap              Other    2015-
## 5 83732         2        1     4 94000100   240 Water, tap              Other    2015-
## 6 83732         2        1     8 94100100   480 Water, bottled, unswe... Grocery  2015-
## # i 2 more variables: Foodgroup_FNDDS <chr>, ratio_FAFH_FAH <dbl>

```

```

both_days4 %>% filter(is.na(ratio_FAFH_FAH)) %>% select(Foodgroup_FNDDS) %>% disti

```

```

## # A tibble: 2 × 1
##   Foodgroup_FNDDS
##   <chr>
## 1 water
## 2 babyfood

```

```
# assign ratio to 1 for water and babyfood just as a placeholder
both_days5 <- both_days4 %>%
  mutate(ratio_FAFH_FAH = ifelse(Foodgroup_FNDDS == "water" | Foodgroup_FNDDS == "
```

We see that all missing ratios are for water or babyfood, which we're not using in this analysis, so we can ignore and just assign 1.

Do a final check.

```
both_days5 %>% filter(is.na(ratio_FAFH_FAH)) # none-good
```

```
## # A tibble: 0 × 14
## # i 14 variables: seqn <dbl>, daysintake <dbl>, reliable <dbl>, line <dbl>, foodcode <db
## #   description <chr>, foodsource <chr>, nhanes_cycle <chr>, dayrec <dbl>, added_sugar <
## #   Foodgroup_FNDDS <chr>, ratio_FAFH_FAH <dbl>
```

Food Cost Data

There are two cost datasets: one for the years 2015-2016 and the other for the years 2017-2018. We need to merge these with the corresponding NHANES cohorts (2015-2016 and 2017-2018).

First import and join the two datasets. Then, because the variable “price_100g” represents the food cost per 100 grams of food, we create a new variable that represents the food cost per 1 gram of food (which is what we want to use later on).

```

# import 2015-2016 price data
cost1516 <- read_xlsx("data_inputs/ECONOMIC/food_prices/DATA/pp_national_average_p
                      sheet = "PP-NAP1516") %>%
  select(food_code, price_100gm) %>%
  mutate(nhanes1516 = 1)

# import 2017-2018
cost1718 <- read_xlsx("data_inputs/ECONOMIC/food_prices/DATA/pp_national_average_p
                      sheet = "PP-NAP1718") %>%
  select(food_code, price_100gm) %>%
  mutate(nhanes1516 = 0)

# combine price datasets
price_comb <- rbind(cost1516, cost1718) %>%
  mutate(price_per_gram = price_100gm / 100) # create price_per_gram var

```

Then, join with both_days and check for missing values. We also calculate two variables, two_digits and three_digits, that represent the first two and three digits, respectively, of the NHANES FNDDS food code. The beginning digits of the food code correspond with different food categorizations created by the USDA. These variables will be used later.

```

# create new nhanes cycle variable
both_days6 <- both_days5 %>%
  mutate(nhanes1516 = ifelse(nhanes_cycle == "2015-2016", 1, 0))

# join with food data
both_days7 <- left_join(both_days6, price_comb, by = c("foodcode" = "food_code", "
  mutate(foodcode = as.character(foodcode),
         two_digits = as.numeric(substr(foodcode, 1, 2)), #first two digits of fnc
         three_digits = as.numeric(substr(foodcode, 1, 3))) #first three digits of

# how many are missing price data?
both_days7 %>% filter(is.na(price_per_gram)) %>% head()

```

```
## # A tibble: 6 × 19
##   seqn daysintake reliable   line foodcode grams description      foodsource nhanes
##   <dbl>      <dbl>    <dbl> <dbl> <chr>    <dbl> <chr>          <chr>    <chr>
## 1 83734          2        1     4 27510233  225 Cheeseburger, 1 mediu... Other    2015-
## 2 83735          1        1     3 27540250  242 Chicken fillet, broil... Other    2015-
## 3 83737          2        1     1 55200070   80 Waffle, whole grain, ... Grocery  2015-
## 4 83737          2        1     1 32130830   77 Egg omelet or scrambl... Grocery  2015-
## 5 83742          2        1    15 58146391  133. Pasta with cream sauc... Other    2015-
## 6 83742          2        1    16 58146301  133. Pasta with tomato-bas... Other    2015-
## # i 7 more variables: Foodgroup_FNDDS <chr>, ratio_FAFH_FAH <dbl>, nhanes1516 <dbl>, pri
## #   price_per_gram <dbl>, two_digits <dbl>, three_digits <dbl>
```

```
both_days7 %>% filter(is.na(price_100gm)) %>%
  select(foodcode, description, Foodgroup_FNDDS) %>%
  distinct() %>%
  head()
```

```
## # A tibble: 6 × 3
##   foodcode description
##   <chr>    <chr>
## 1 27510233 Cheeseburger, 1 medium patty, with condiments, on bun, from fast food / resta
## 2 27540250 Chicken fillet, broiled, sandwich with cheese, on whole wheat roll, with lett
## 3 55200070 Waffle, whole grain, reduced fat, from frozen
## 4 32130830 Egg omelet or scrambled egg, with meat and dark-green vegetables, fat added i
## 5 58146391 Pasta with cream sauce and added vegetables, restaurant
## 6 58146301 Pasta with tomato-based sauce, and added vegetables, restaurant
```

There are a fair number of **missing values**, so we are going to calculate the average price for each food group (determined by first two digits of FNDDS food code) x foodsource (grocery vs not) combination, and then use these food group-level averages for the missing values (i.e., impute).

Create a dataset called `imputed_price` that contains the average prices for each food group x food source combination.

```
imputed_price <-
  both_days7 %>%
  select(seqn, line, foodcode, two_digits, foodsource, price_per_gram) %>%
  distinct() %>%
  group_by(two_digits, foodsource) %>%
  summarise(price_g_group_mean = mean(price_per_gram, na.rm = TRUE),
            price_g_group_median = median(price_per_gram, na.rm = TRUE),
            price_g_group_sd = sd(price_per_gram, na.rm = TRUE))
```

``summarise()`` has grouped output by 'two_digits'. You can override using the ``.groups`` a

Then, merge with `both_days` and calculate a new variable, `price_per_gram`, that replaces any missing price value with the group  median.

```
# join imputed prices with food data
both_days8 <- left_join(both_days7, imputed_price, by = c("two_digits", "foodsource"))

# insert food_group median price if missing
both_days9 <- both_days8 %>%
  mutate(price_per_gram = ifelse(is.na(price_per_gram), price_g_group_median, price_per_gram))
ungroup()

# check if any missing price per gram (shouldn't be)
both_days9 %>% filter(is.na(price_per_gram)) #none-good
```

```
## # A tibble: 0 × 22
## # i 22 variables: seqn <dbl>, daysintake <dbl>, reliable <dbl>, line <dbl>, foodcode <ch
## #   description <chr>, foodsource <chr>, nhanes_cycle <chr>, dayrec <dbl>, added_sugar <
## #   Foodgroup_FNDDS <chr>, ratio_FAFH_FAH <dbl>, nhanes1516 <dbl>, price_100gm <dbl>, pr
## #   two_digits <dbl>, three_digits <dbl>, price_g_group_mean <dbl>, price_g_group_mediar
```

Tidy up the global environment.

```
rm(list=setdiff(ls(), c("both_days9", "my_date")))
```

Mixed Dishes

We want to obtain the food price of whole foods that correspond with the food groups in the DGA dietary patterns. Therefore, we do not want to use the prices of “mixed foods” that typically represent larger dishes that incorporate many different food groups and is thus not a good representation of whole foods.

To deal with this, we use a dataset that identifies FNDDS food code prefixes that represent “mixed dishes”. We use this data to categorize food codes into mixed vs. non-mixed foods, and then later stratify our results by this categorization. See **Section XX** to see how these mixed dish codes were determined.

```
mixed <- read_csv("data_inputs/DIET/dietary_intake/DATA/raw_data/mixed/mixed_dishes.csv")
select(-foodcode_desc)
```

```
## Rows: 13 Columns: 3
## — Column specification —————
## Delimiter: ","
## chr (1): foodcode_desc
## dbl (2): foodcode_prefix, mixed_dish
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Some of the categorizations are for prefixes with two digits, and some are for prefixes with three digits.

```
mixed_2dig <- mixed %>% filter(!(foodcode_prefix %in% c(416, 418, 419, 423)))
mixed_3dig <- mixed %>% filter(foodcode_prefix %in% c(416, 418, 419, 423)) %>% rer
```

First, join both_days with the two digits, and then the three digits. Then, create an indicator variable called “mixed_dish” that we can stratify by later.

```
both_days10 <- both_days9 %>%
  left_join(mixed_2dig, by = c("two_digits" = "foodcode_prefix"))

both_days11 <- both_days10 %>%
  rows_patch(mixed_3dig, unmatched = "ignore") %>%
  mutate(mixed_dish = ifelse(is.na(mixed_dish), 0, mixed_dish))
```

```
## Matching, by = "three_digits"
```

Check if everything merged correctly.

```
both_days11 %>% filter(mixed_dish == 1) %>% select(two_digits) %>% distinct() %>%
```

```
## # A tibble: 11 × 1
##   two_digits
##   <dbl>
## 1      27
## 2      28
## 3      32
## 4      33
## 5      41
## 6      42
## 7      58
## 8      59
## 9      76
## 10     77
## 11     78
```

Looks good

Lastly, save a copy of this temporary dataset in case we need it for debugging later.

```
saveRDS(both_days11, "data_inputs/IMPACT_FACTORS/temp_data/both_days11_cost.rds")
```



Tidy up the global environment.

```
rm(list=setdiff(ls(), c("both_days11", "my_date")))
```

5.1.2 Calculate Average Food Cost, Per FNDDS Food Code

Now, we will calculate the average food cost per 1 gram for each FNDDS food code. This process includes aggregating and summarizing the diet dataset.

```
# keep this commented out unless you are starting the script from here
# both_days11 <- read_rds("data_inputs/IMPACT_FACTORS/temp_data/both_days11_cost.r
# my_date <- Sys.Date()
```

First, select the variables we need to eventually calculate the cost impact factors.

```
my_price_table <- both_days11 %>%
  select(seqn, dayrec, line, foodcode, description, Foodgroup_FNDDS,
         foodsource, ratio_FAFH_FAH, price_per_gram, mixed_dish, grams) %>%
  distinct() %>%
  arrange(seqn, dayrec, line)
```

Then, import the **food-level** consumed, inedible, and wasted amounts that was calculated in the previous **Section XXX** and join with my_price table. These values were calculated in the previous section because the inedible and wasted coefficients are at the FCID-level, which is the same level that the environmental impacts are at.

```
# import
fndds_flw <- read_rds("data_inputs/IMPACT_FACTORS/temp_data/fndds_flw.rds") %>%
  mutate(foodcode = as.character(foodcode))

# join
my_price_table1 <- left_join(my_price_table, fndds_flw, by = c("seqn", "dayrec", "

# Look at missing
my_price_table1 %>% filter(is.na(inedible_amt_FNDDS) | is.na(wasted_amt_FNDDS))
```

```
## # A tibble: 0 × 14
## # i 14 variables: seqn <dbl>, dayrec <dbl>, line <dbl>, foodcode <chr>, description <chr>
## #   foodsource <chr>, ratio_FAFH_FAH <dbl>, price_per_gram <dbl>, mixed_dish <dbl>, gran
## #   consumed_amt_FNDDS <dbl>, inedible_amt_FNDDS <dbl>, wasted_amt_FNDDS <dbl>
```

Now, we need to adjust for the difference in price for food purchased at the grocery store vs. not purchased there. We do this by multiplying the food-level FAFH-FAH ratio by the food price/1g rate if the food source is “Other” (i.e., not grocery).

We also need to calculate the total price for the amounts of food consumed (price_impact_per_foodcode_Consumed).

```
# adjust price for FAFH
# if food source is 'other', then multiple price by fah/fafh ratio
my_price_table2 <- my_price_table1 %>% mutate(price_per_gram_adjusted = ifelse(foc
                                          price_per_gram * ratio_
                                          price_per_gram),
                                          price_impact_per_foodcode_Consumed =
```

Now, we want to calculate the total amount of money spent on each FNDDS foodcode, per person, per day. This is necessary because for some participants, they consumed a given FNDDS foodcode multiple times in the same day, and we want to combine these so each FNDDS food code only has one corresponding daily price. For example, if someone had 100 grams of coffee in the morning and then another 75 grams in the afternoon, we want to get the price for 175 grams of coffee consumed that day.

```
# Calculate total price per day, per person
price_impact_total <- my_price_table2 %>%
  group_by(seqn, foodcode, dayrec, mixed_dish) %>%
  summarise(price_per_day_Consumed = sum(price_impact_per_foodcode_Consumed),
            consumed_per_day = sum(consumed_amt_FNDDS),
            inedible_per_day = sum(inedible_amt_FNDDS),
            wasted_per_day = sum(wasted_amt_FNDDS))

## `summarise()` has grouped output by 'seqn', 'foodcode', 'dayrec'. You can override using
```

Transform dataset to wide format.

```
price_wide_total <- pivot_wider(price_impact_total,
                                names_from = c(dayrec),
                                values_from = c(price_per_day_Consumed,
                                                  consumed_per_day,
                                                  inedible_per_day,
                                                  wasted_per_day))
```

Because some participants have 2 days of dietary recall, we need to calculate an average price for each foodcode, per participant. If a participant only has 1 day of recall, then we use that to represent the average.

To calculate the average, we first need to determine how many days of recall each participant has.

```
# calculate # days of recall
both_days11 %>% select(reliable) %>% table()
```

```
## reliable
##      1      4
## 420721  7623
```

```
daysofintake <- both_days11 %>% select(seqn, daysintake, reliable) %>% distinct()
```

```
# join
price_wide_total1 <- price_wide_total %>%
  left_join(daysofintake, by = "seqn")
```

```
## Warning in left_join(., daysofintake, by = "seqn"): Detected an unexpected many-to-many
## i Row 10171 of `x` matches multiple rows in `y`.
## i Row 1 of `y` matches multiple rows in `x`.
## i If a many-to-many relationship is expected, set `relationship = "many-to-many"` to sil
```

```
# check missing
price_wide_total1 %>% filter(daysintake == 1 & is.na(consumed_per_day_1)) %>% head
```

```
## # A tibble: 6 × 13
## # Groups:   seqn, foodcode [6]
##   seqn foodcode mixed_dish price_per_day_Consumed_2 price_per_day_Consumed_1 consumed_r
##   <dbl> <chr>      <dbl>          <dbl>          <dbl>          <dbl>
## 1 84306 11000000      0             NA             NA
## 2 85182 11000000      0             NA             NA
## 3 85616 11000000      0             NA             NA
## 4 86025 11000000      0             NA             NA
## 5 86408 11000000      0             NA             NA
## 6 86704 11000000      0             NA             NA
## # i 6 more variables: inedible_per_day_2 <dbl>, inedible_per_day_1 <dbl>, wasted_per_day
## #   wasted_per_day_1 <dbl>, daysintake <dbl>, reliable <dbl>
```

Now, we have to fill in some 0s to properly calculate averages later on.

If a participant has 2 days of intake and their corresponding price for day 1 or day 2 is missing (NA), then we need to replace those NAs with 0s, because in this case, the data aren't "missing", the participant just didn't consume/spend money on that food that day.


```

# add 0s when appropriate
price_wide_total2 <- price_wide_total1 %>%
  mutate(
    # day 2
    price_per_day_Consumed_2 = ifelse(daysintake == 2 & is.na(price_per_day_Consumed_2), 0, price_per_day_Consumed_2),
    consumed_per_day_2 = ifelse(daysintake == 2 & is.na(consumed_per_day_2), 0, consumed_per_day_2),
    wasted_per_day_2 = ifelse(daysintake == 2 & is.na(wasted_per_day_2), 0, wasted_per_day_2),
    inedible_per_day_2 = ifelse(daysintake == 2 & is.na(inedible_per_day_2), 0, inedible_per_day_2),

    # day1
    price_per_day_Consumed_1 = ifelse(daysintake == 1 & is.na(price_per_day_Consumed_1), 0, price_per_day_Consumed_1),
    consumed_per_day_1 = ifelse(daysintake == 1 & is.na(consumed_per_day_1), 0, consumed_per_day_1),
    wasted_per_day_1 = ifelse(daysintake == 1 & is.na(wasted_per_day_1), 0, wasted_per_day_1),
    inedible_per_day_1 = ifelse(daysintake == 1 & is.na(inedible_per_day_1), 0, inedible_per_day_1)
  )

```

Calculate the **average** total price of consumed food at the FNDDS foodcode-level for each person. This will give us the data we need to calculate the cost impact factors in the next section.

```

price_wide_total3 <- price_wide_total2 %>%
  ungroup() %>%
  rowwise() %>%
  mutate(
    price_total_avg_Consumed = mean(c(price_per_day_Consumed_1, price_per_day_Consumed_2), na.rm = TRUE),
    consumed_total_avg = mean(c(consumed_per_day_1, consumed_per_day_2), na.rm = TRUE),
    inedible_total_avg = mean(c(inedible_per_day_1, inedible_per_day_2), na.rm = TRUE),
    wasted_total_avg = mean(c(wasted_per_day_1, wasted_per_day_2), na.rm = TRUE)
  )
  select(seqn, foodcode, mixed_dish, price_total_avg_Consumed,
         consumed_total_avg, inedible_total_avg, wasted_total_avg)

```

We also want to calculate the same thing as above, but additionally stratified by food source (grocery vs. non-grocery). So we do the same calculations above, but additionally group by “foodsource”.

```
# STRATIFIED BY FAH VS FAFH
# Calculate fah and fafh price, per person
price_impact_split <- my_price_table2 %>%
  group_by(seqn, foodcode, dayrec, foodsource, mixed_dish) %>%
  summarise(price_per_day_Consumed = sum(price_impact_per_foodcode_Consumed),
            consumed_per_day = sum(consumed_amt_FNDDS),
            inedible_per_day = sum(inedible_amt_FNDDS),
            wasted_per_day = sum(wasted_amt_FNDDS))
```

`summarise()` has grouped output by 'seqn', 'foodcode', 'dayrec', 'foodsource'. You can
argument.

Calculate the average of Day 1 and Day 2 prices for each FNDDS foodcode.

```
price_wide_split <- pivot_wider(price_impact_split,
                                names_from = c(foodsource, dayrec),
                                values_from = c(price_per_day_Consumed,
                                                  consumed_per_day,
                                                  inedible_per_day,
                                                  wasted_per_day))

# join with days of intake dataset
price_wide_split1 <- price_wide_split %>%
  left_join(daysofintake, by = "seqn")
```

Warning in left_join(., daysofintake, by = "seqn"): Detected an unexpected many-to-many
i Row 10171 of `x` matches multiple rows in `y`.
i Row 1 of `y` matches multiple rows in `x`.
i If a many-to-many relationship is expected, set `relationship = "many-to-many"` to sil

```
# check
```

```
price_wide_split1 %>% filter(daysintake == 1 & is.na(consumed_per_day_Other_1)) #g
```

```
## # A tibble: 17,721 × 21
```

```
## # Groups:   seqn, foodcode [17,721]
```

```
##   seqn foodcode mixed_dish price_per_day_Consum...1 price_per_day_Consum...2 price_per_day
```

```
##   <dbl> <chr>          <dbl>          <dbl>          <dbl>
```

```
##  1 83735 12310350      0             NA             0.0719
```

```
##  2 83735 54301100      0             NA             0.161
```

```
##  3 83735 58106602      1             NA             0.725
```

```
##  4 83735 71405040      0             NA             0.355
```

```
##  5 83735 91705300      0             NA             0.131
```

```
##  6 83735 91705310      0             NA             0.253
```

```
##  7 83735 91745010      0             NA             0.0791
```

```
##  8 83735 92410370      0             NA             0.521
```

```
##  9 83735 94100100      0             NA             0.0634
```

```
## 10 83750 11112110      0             NA             0.458
```

```
## # i 17,711 more rows
```

```
## # i abbreviated names: 1price_per_day_Consumed_Other_2, 2price_per_day_Consumed_Grocery_
```

```
## #   3price_per_day_Consumed_Grocery_2, 4price_per_day_Consumed_Other_1
```

```
## # i 14 more variables: consumed_per_day_Other_2 <dbl>, consumed_per_day_Grocery_1 <dbl>,
```

```
## #   consumed_per_day_Grocery_2 <dbl>, consumed_per_day_Other_1 <dbl>, inedible_per_day_C
```

```
## #   inedible_per_day_Grocery_1 <dbl>, inedible_per_day_Grocery_2 <dbl>, inedible_per_day
```

```
## #   wasted_per_day_Other_2 <dbl>, wasted_per_day_Grocery_1 <dbl>, wasted_per_day_Grocery
```

```

# add 0s when appropriate
price_wide_split2 <- price_wide_split1 %>%
  mutate(

    # TWO DAYS OF INTAKE

    # day 2 - Other
    price_per_day_Consumed_Other_2 = ifelse(daysintake == 2 & is.na(price_per_day_
    consumed_per_day_Other_2 = ifelse(daysintake == 2 & is.na(consumed_per_day_Oth
    wasted_per_day_Other_2 = ifelse(daysintake == 2 & is.na(wasted_per_day_Other_2
    inedible_per_day_Other_2 = ifelse(daysintake == 2 & is.na(inedible_per_day_Oth

    # day1 - Other
    price_per_day_Consumed_Other_1 = ifelse(daysintake == 2 & is.na(price_per_day_
    consumed_per_day_Other_1 = ifelse(daysintake == 2 & is.na(consumed_per_day_Oth
    wasted_per_day_Other_1 = ifelse(daysintake == 2 & is.na(wasted_per_day_Other_1
    inedible_per_day_Other_1 = ifelse(daysintake == 2 & is.na(inedible_per_day_Oth

    # day 2 - Grocery
    price_per_day_Consumed_Grocery_2 = ifelse(daysintake == 2 & is.na(price_per_da
    consumed_per_day_Grocery_2 = ifelse(daysintake == 2 & is.na(consumed_per_day_G
    wasted_per_day_Grocery_2 = ifelse(daysintake == 2 & is.na(wasted_per_day_Groce
    inedible_per_day_Grocery_2 = ifelse(daysintake == 2 & is.na(inedible_per_day_G

    # day1 - Grocery
    price_per_day_Consumed_Grocery_1 = ifelse(daysintake == 2 & is.na(price_per_da
    consumed_per_day_Grocery_1 = ifelse(daysintake == 2 & is.na(consumed_per_day_G
    wasted_per_day_Grocery_1 = ifelse(daysintake == 2 & is.na(wasted_per_day_Groce
    inedible_per_day_Grocery_1 = ifelse(daysintake == 2 & is.na(inedible_per_day_G

    # ONE DAY OF INTAKE

    # day1 - Other
    price_per_day_Consumed_Other_1 = ifelse(daysintake == 1 & is.na(price_per_day_
    consumed_per_day_Other_1 = ifelse(daysintake == 1 & is.na(consumed_per_day_Oth

```

```

wasted_per_day_Other_1 = ifelse(daysintake == 1 & is.na(wasted_per_day_Other_1)
inedible_per_day_Other_1 = ifelse(daysintake == 1 & is.na(inedible_per_day_Other_1)

# day1 - Grocery
price_per_day_Consumed_Grocery_1 = ifelse(daysintake == 1 & is.na(price_per_day_Consumed_Grocery_1), 0, price_per_day_Consumed_Grocery_1)
consumed_per_day_Grocery_1 = ifelse(daysintake == 1 & is.na(consumed_per_day_Grocery_1), 0, consumed_per_day_Grocery_1)
wasted_per_day_Grocery_1 = ifelse(daysintake == 1 & is.na(wasted_per_day_Grocery_1), 0, wasted_per_day_Grocery_1)
inedible_per_day_Grocery_1 = ifelse(daysintake == 1 & is.na(inedible_per_day_Grocery_1), 0, inedible_per_day_Grocery_1)

# summarize at foodcode-level for each person
price_wide_split3 <- price_wide_split2 %>%
  ungroup() %>%
  rowwise() %>%
  mutate(
    #FAH (Grocery)
    price_fah_avg_Consumed = mean(c(price_per_day_Consumed_Grocery_1, price_per_day_Consumed_Other_1), na.rm=T),
    consumed_fah_avg = mean(c(consumed_per_day_Grocery_1, consumed_per_day_Grocery_2), na.rm=T),
    inedible_fah_avg = mean(c(inedible_per_day_Grocery_1, inedible_per_day_Grocery_2), na.rm=T),
    wasted_fah_avg = mean(c(wasted_per_day_Grocery_1, wasted_per_day_Grocery_2), na.rm=T),

    #FAFH (Other)
    price_fafh_avg_Consumed = mean(c(price_per_day_Consumed_Other_1, price_per_day_Consumed_Grocery_1), na.rm=T),
    consumed_fafh_avg = mean(c(consumed_per_day_Other_1, consumed_per_day_Grocery_1), na.rm=T),
    inedible_fafh_avg = mean(c(inedible_per_day_Other_1, inedible_per_day_Grocery_1), na.rm=T),
    wasted_fafh_avg = mean(c(wasted_per_day_Other_1, wasted_per_day_Grocery_1), na.rm=T),

    select(seqn, foodcode, mixed_dish,
           price_fah_avg_Consumed,
           consumed_fah_avg, inedible_fah_avg, wasted_fah_avg,
           price_fafh_avg_Consumed,
           consumed_fafh_avg, inedible_fafh_avg, wasted_fafh_avg)
  )

```

Join the total price and the **split** price averages into one dataset.

```
# join
```

```
price_wide_comb <- full_join(price_wide_total3, price_wide_split3, by = c("seqn",
```

```
## Warning in full_join(price_wide_total3, price_wide_split3, by = c("seqn", : Detected an  
## i Row 10171 of `x` matches multiple rows in `y`.  
## i Row 10171 of `y` matches multiple rows in `x`.  
## i If a many-to-many relationship is expected, set `relationship = "many-to-many"` to sil
```

```

# create foodcode-to-food group "mapping" ← join with
map <- both_days11 %>%
  ungroup() %>%
  select(foodcode, Foodgroup_FNDDS) %>%
  distinct()

# join
price_wide_comb1 <- left_join(price_wide_comb, map, by = "foodcode") %>%
  relocate(Foodgroup_FNDDS, .after = mixed_dish)

# add up prices by food group
price_wide_comb2 <- price_wide_comb1 %>%
  group_by(seqn, Foodgroup_FNDDS, mixed_dish) %>%
  summarise(
    # Consumed
    price_total_sum_Consumed = sum(price_total_avg_Consumed),
    consumed_total_sum = sum(consumed_total_avg),

    price_fah_sum_Consumed = sum(price_fah_avg_Consumed),
    consumed_fah_sum = sum(consumed_fah_avg),

    price_fafh_sum_Consumed = sum(price_fafh_avg_Consumed),
    consumed_fafh_sum = sum(consumed_fafh_avg),

    # Inedible
    inedible_total_sum = sum(inedible_total_avg),
    inedible_fah_sum = sum(inedible_fah_avg),
    inedible_fafh_sum = sum(inedible_fafh_avg),

    # Wasted
    wasted_total_sum = sum(wasted_total_avg),
    wasted_fah_sum = sum(wasted_fah_avg),
    wasted_fafh_sum = sum(wasted_fafh_avg))

```

`summarise()` has grouped output by 'seqn', 'Foodgroup_FNDDS'. You can override using `tr`

```
# transform to wide
price_wide_comb3 <- price_wide_comb2 %>%
  pivot_wider(id_cols = seqn,
              names_from = c(Foodgroup_FNDDS, mixed_dish),
              values_from = -c(seqn, Foodgroup_FNDDS, mixed_dish)) %>%
  replace(is.na(.), 0)
```

Export dataset to the temporary folder, so it can be used to calculate the impact factors in the next section.

```
write_rds(price_wide_comb3, "data_inputs/IMPACT_FACTORS/temp_data/price_input_dat.
```